

이음 프로젝트의

SW 시험 계획서(STP)

2026. 05. 03

팀장 정승민
팀원 김수빈
김재현
최정원

이음

SW 시험 계획서(STP)

목차

1. 개요

- 1.1 목적 및 범위
- 1.2 테스트 환경

2. 테스트 계획 수립

- 2.1 비기능 테스트 계획
- 2.2 단위 테스트 계획
- 2.3 통합 테스트 계획
- 2.4 시스템 테스트 계획
- 2.5 인수 테스트 계획

1. 개요

1.1 목적 및 범위

본 문서는 이음(Eeum) 시스템의 시험 활동을 계획하고, 시험 환경, 시험 종류별 범위, 시험 케이스 작성 기준을 정의한다. SDD에 명시된 모든 기능과 비기능 요구사항이 정상 동작함을 검증하는 것을 목적으로 한다.

본 시험 계획서의 적용 범위는 다음과 같다:

- 비기능 시험 (성능, 보안, 가용성, 확장성)
- 단위 시험 (Service Layer 메서드 단위)
- 통합 시험 (Controller ↔ Service ↔ Repository ↔ DB)
- 시스템 시험 (전체 시나리오 기반 End-to-End)
- 인수 시험 (사용자 시나리오 기반 검수)

1.2 테스트 환경

(1) 하드웨어 환경

개발 환경(로컬)	개발자 PC
통합 테스트 환경	GitHub Actions Ubuntu Runner + Testcontainers
시스템 테스트 환경	AWS EC2 t2.micro (스테이징, 프리티어)
부하 테스트 환경	별도 EC2 + JMeter Client

(2) 소프트웨어 환경

분류	도구
백엔드 단위 / 통합 테스트	JUnit 5, Mockito, AssertJ, Testcontainers
프론트엔드 단위 테스트	Jest, React Testing Library
API 테스트	Postman, REST Assured
부하 테스트	Apache JMeter, k6
보안 테스트	OWASP ZAP, Burp Suite Community
커버리지 측정	JaCoCo (백엔드), Jest Coverage (프론트엔드)
테스트 DB	MySQL 8.0 (Testcontainers)
테스트 캐시	Redis 7 (Testcontainers)

(3) 테스트 데이터

- 개발 환경: Faker 라이브러리로 생성된 더미 데이터
- 통합 테스트: 각 테스트마다 Testcontainers로 격리된 DB
- 시스템 테스트: 사전 정의된 시드 스크립트 (회원/매장/상품 각 10건 이상)

2. 테스트 계획 수립

2.1 비기능 테스트 계획

비기능 시험은 시스템의 품질 속성(성능, 보안, 가용성)이 요구사항을 만족하는지 검증한다.

2.1.1 성능 시험

항목	목표	측정 방법
API 응답 시간(p95)	500ms 이하	JMeter, Actuator Metrics
API 응답 시간(p99)	1초 이하	JMeter, Actuator Metrics
동시 사용자 수	100명 이상	JMeter Ramp-up
TPS(Transactions Per Second)	50 TPS 이상	JMeter Throughput
DB 쿼리 시간	평균 50ms 이하	Slow Query Log
Redis 응답 시간	평균 10ms 이하	Redis SLOWLOG

시험 시나리오:

- 시나리오 1: 매장 검색 100명 동시 호출
- 시나리오 2: 주문 생성 50명 동시 호출 (재고 차감 동시성)
- 시나리오 3: 채팅 메시지 발송 200명 동시 (WebSocket)
- 시나리오 4: 로그인 100명 동시 (JWT 발급)

2.1.2 보안 시험

항목	검증 방법
JWT 토큰 만료 검증	만료된 토큰으로 요청 시 401 응답 반환 확인
권한 분리	USER 권한으로 /owner/**, /admin/** 접근 시 403 응답 반환 확인
SQL Injection	OWASP ZAP 자동 스캔을 통해 SQL Injection 취약점 여부 확인
XSS	사용자 입력값 이스케이프 처리 및 스크립트 실행 차단 여부 확인
CSRF	Stateless JWT 기반 인증 구조에서 CSRF 비활성화 처리 여부 확인
비밀번호 해싱	BCrypt 적용 여부 확인 및 DB에 평문 비밀번호가 저장되지 않는지 검증
Webhook 서명 검증	잘못된 서명으로 Webhook 호출 시 401 응답 반환 확인
민감정보 마스킹	API 응답 및 로그에 민감정보가 마스킹되어 출력되는지 확인
Re-auth 토큰 1회용 검증	동일 Re-auth 토큰을 2회 사용 시 401 응답 반환 확인

2.1.3 가용성 시험

항목	목표/검증 방법
시스템 가동률	99% 이상
평균 복구 시간(MTTR)	30분 이하
헬스 체크 응답	/actuator/health 요청 시 200 응답 반환
외부 API 장애 대응	Circuit Breaker 동작 여부 확인

2.1.4 확장성 시험 (참고)

프리티어 환경에서는 수평 확장이 제한되므로, 향후 확장 시 검증 항목을 정의한다:

- EC2 Auto Scaling 동작
- RDS Read Replica 라우팅
- ElastiCache Multi-AZ 페일오버

2.2 단위 테스트 계획

2.2.1 단위 테스트 범위

- Service Layer 메서드: 비즈니스 로직 검증 (목표 커버리지 70%)
- Entity 메서드: 상태 전이, 검증 메서드 (목표 100%)
- Util/Helper 클래스: 유틸 함수 (목표 90%)
- AOP Aspect: @AdminAudit, @DistributedLock 등 (목표 80%)

2.2.2 작성 규칙

- 테스트 클래스명: {Class}Test (예: OrderServiceTest)
- 테스트 메서드명: 한글 + 백틱 사용 가능 (예: `재고가 부족하면 예외를 발생시킨다`())
- Given-When-Then 구조 준수
- 외부 의존성은 모두 Mockito로 Mocking
- 한 메서드당 검증은 1개 시나리오 (단일 책임)

2.2.3 주요 테스트 시나리오 예시

Service	메서드	시나리오
AuthService	signup()	정상 가입 / 이메일 중복 / 닉네임 중복 / 인증 미완료
AuthService	login()	정상 로그인 / 비밀번호 불일치 / 탈퇴 회원 / 정지 회원
OrderService	createOrder()	정상 주문 / 재고 부족 / 비활성 상품 / 락 획득 실패
OrderService	cancelOrder()	정상 취소 / 이미 취소된 주문 / 취소 불가 상태
ProductOptionService	validateAndCalculatePrice()	정상 계산 / 필수 그룹 미선택 / 비활성 선택지 / 다른 상품 선택지
FavoriteService	toggleFavorite()	처음 찜 / 이미 찜 상태에서 해제 / 카운트 증감 검증
ReservationService	createReservation()	정상 예약 / 분 단위 capacity 초과

2.2.4 단위 테스트 예시 (Java)

@ExtendWith(MockitoExtension.class)

class OrderServiceTest {

 @InjectMocks private OrderService orderService;

 @Mock private OrderRepository orderRepository;

 @Mock private ProductRepository productRepository;

 @Test

void `재고가 부족하면 예외를 발생시킨다`() {

 // Given

 Long accountId = 1L;

 OrderCreateRequestDto dto = ...;

 Product product = Product.builder().stock(5).build();

 given(productRepository.findByIdForUpdate(anyLong()))

 .willReturn(Optional.of(product));

 // When & Then

 assertThatThrownBy(() -> orderService.createOrder(accountId, dto))

 .isInstanceOf(BusinessException.class)

 .hasMessageContaining("재고");

 }

}

2.3 통합 테스트 계획

2.3.1 통합 테스트 범위

- Controller ↔ Service ↔ Repository ↔ MySQL/Redis 전체 흐름
- 외부 API는 Mock Server (WireMock) 또는 Stub 사용
- 트랜잭션 동작 검증 (롤백, AFTER_COMMIT 이벤트)
- 멱등성 시나리오 (Webhook 중복 수신, Idempotency-Key)
- 분산 락 동작 (동시 요청 시 직렬화)

2.3.2 사용 도구

- Testcontainers: MySQL 8.0, Redis 7 컨테이너 자동 기동
- Spring Boot Test: @SpringBootTest, @DataJpaTest
- MockMvc: Controller 통합 테스트
- WireMock: PortOne, FCM 등 외부 API 스텝

2.3.3 주요 통합 테스트 시나리오

시나리오 ID	내용
IT-AUTH-001	회원가입 → 이메일 인증 → 로그인 전체 흐름 검증
IT-AUTH-002	로그인 → 토큰 만료 → 재발급 흐름 검증
IT-ORD-001	장바구니 → 주문 생성 → PortOne Mock → 결제 완료 흐름 검증
IT-ORD-002	주문 생성 → 사장 확정 → 사용자 알림 발송 검증
IT-ORD-003	동시 100명이 동일 상품 주문 시 재고가 정확히 차감되는지 검증
IT-ORD-004	PortOne Webhook을 동일 ID로 2회 호출해도 1회만 처리되는지 검증
IT-RES-001	같은 시간 슬롯에 동시 예약 요청 시 capacity 초과가 차단되는지 검증
IT-CHAT-001	1:1 채팅방 동시 생성 요청 시 채팅방이 1개만 생성되는지 검증
IT-CHAT-002	메시지 발송 후 AFTER_COMMIT 이벤트로 unread 수가 증가하는지 검증
IT-NOTI-001	주문 생성 이벤트 발생 시 사장 알림 저장 및 Redis unread 증가 검증
IT-FAV-001	좋아요 토글 100회 수행 후 카운트 정합성 유지 여부 검증

2.4 시스템 테스트 계획

2.4.1 시스템 테스트 범위

사용자 관점에서의 End-to-End 시나리오 검증.

2.4.2 주요 시스템 테스트 시나리오

ST ID	시나리오	검증 내용
ST-001	신규 회원 가입 → 활동지역 등록 → 매장 검색 → 찜	사용자 첫 진입 흐름 검증
ST-002	사장 가입 → 사업자 인증 → 관리자 승인 → 매장 등록	사장 온보딩 흐름 검증
ST-003	상품 등록 → 옵션 추가 → 이벤트 등록 → 사용자 주문	사장 운영 흐름 검증
ST-004	사용자 장바구니 → 주문 → 결제 → 사장 확인 → 완료 → 리뷰	거래 전체 흐름 검증
ST-005	매장 방문 예약 → 사장 확정 → 방문 → 완료	예약 흐름 검증
ST-006	중고상품 등록 → 채팅 문의 → 거래 예약 → 거래 완료 → 리뷰	중고거래 흐름 검증
ST-007	게시글 작성 → 댓글/대댓글 → 좋아요 → 알림 수신	커뮤니티 흐름 검증
ST-008	사용자 신고 → 관리자 검토 → 강제 삭제 → 감사 로그	신고 처리 흐름 검증
ST-009	회원 탈퇴 → 30일 후 자동 삭제	Soft Delete 흐름 검증
ST-010	결제 실패 → 환불 → 재고 복구 → 알림	결제 실패 복구 흐름 검증

2.5 인수 테스트 계획

2.5.1 인수 테스트 목적

실제 사용자 관점에서 요구사항이 충족되었는지 검수한다.

2.5.2 인수 기준

- SRS의 모든 기능 요구사항(FR) 동작 확인
- 비기능 요구사항(NFR) 목표 수치 달성
- UI/UX가 Figma 디자인과 일치
- 주요 시나리오에서 결제/거래/채팅 안정 동작

2.5.3 인수 시험 방법

- 사용자 역할별 테스트: 일반 회원 / 사장 / 관리자
- 체크리스트 기반: 기능별 체크리스트 작성, 통과 여부 기록
- 사용성 평가: 실제 사용자 5명 이상 시범 사용 후 피드백 수집

2.5.4 합격 기준

구분	기준
기능 요구사항	전체 FR 100% 통과
비기능 요구사항	전체 NFR 90% 이상 통과
Critical / Major 결함	0건
Minor 결함	5건 이하. 단, 수정 일정이 합의된 경우 인수 가능